

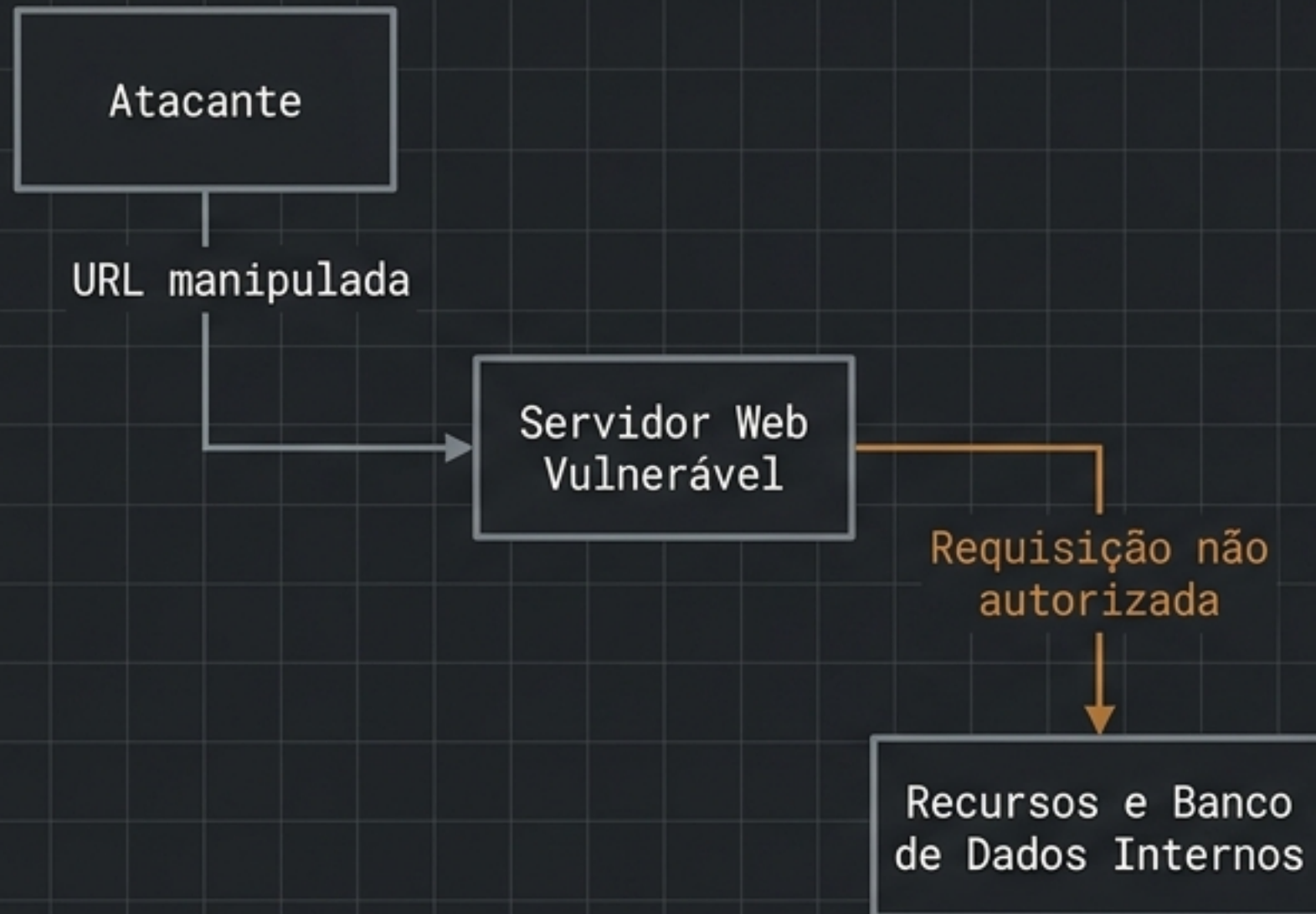
SSRFmap: automação de testes de segurança para vulnerabilidades SSRF

Análise técnica, funcionamento e aplicação em testes de penetração

Pesquisador Chefe: Matheus Ventura
Disciplina: Segurança e teste de software
Data: Junho 2026

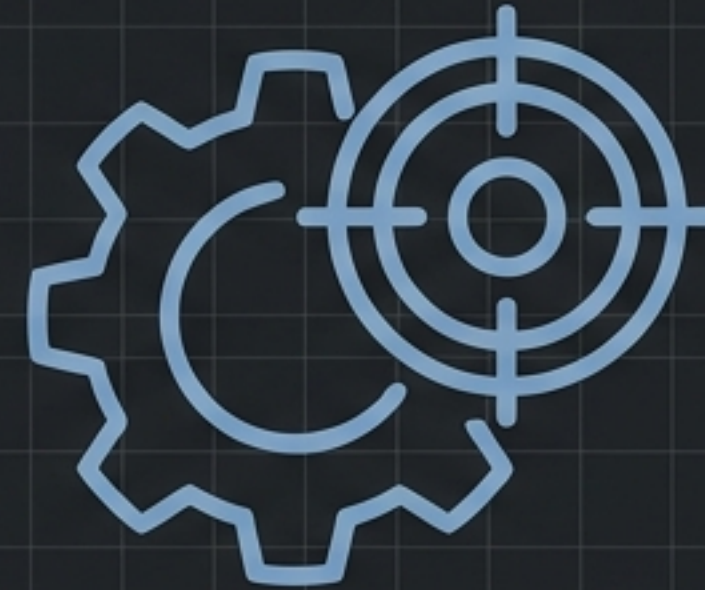
0 Problema vs. A Solução Automatizada

0 Problema: SSRF



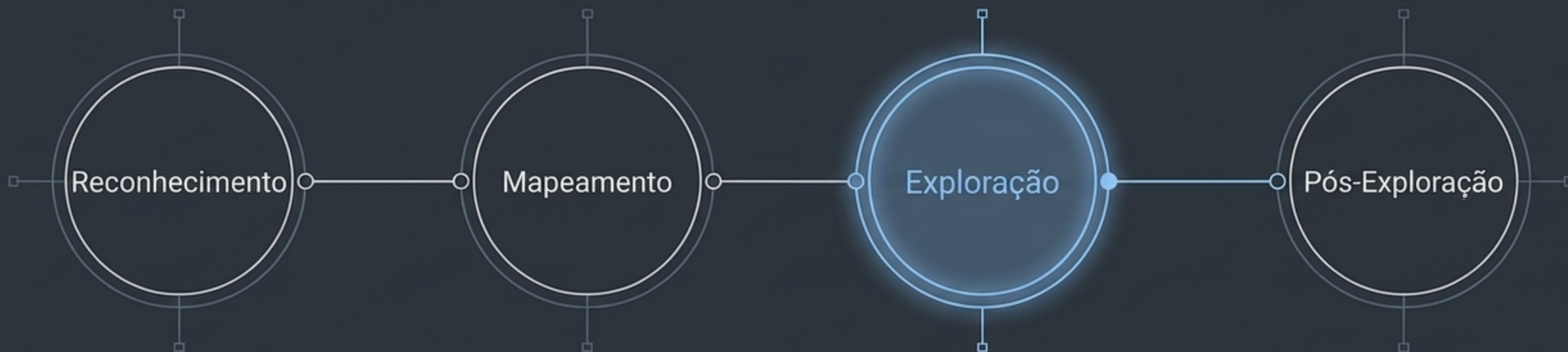
O servidor é enganado, permitindo quebra de isolamento de rede, acesso a recursos locais e extração de metadados em nuvem.

A Solução: SSRFmap



- Framework em Python 3 projetado para fuzzing e exploração profunda.
- Automatiza a descoberta de falhas e a injeção de payloads.
- Atua no SSRF com a mesma eficiência que o SQLmap atua no SQL Injection.

Posicionamento Metodológico



Abordagem

Caixa-preta (Black-box). Foco na manipulação de inputs maliciosos sem a necessidade de acesso ao código-fonte da aplicação.

Alvos Principais

APIs REST e Aplicações Web. Funcionalidades como exportação de PDF, webhooks e pré-visualização de links.

Requisito de Entrada

```
GET /resource HTTP/1.1
Host: example.com
User-Agent: ...
```

Exige uma requisição HTTP previamente interceptada salva em formato texto (.txt) para servir como molde de ataque.

Sintaxe e Funcionamento Básico

```
python3 ssrfmap.py -r req.txt -p url -m portscan --level 2
```

Arquivo contendo a requisição HTTP interceptada.

Parâmetro específico que sofrerá o fuzzing.

Módulo de ataque selecionado.

Nível de evasão. Aplica encodings automáticos para burlar WAFs e filtros de segurança.



SSRFmap

[INFO]:Module 'portscan' launched !

[14:58:12]	IP:127.0.0.1	, Found open	port nº80
[14:58:12]	IP:127.0.0.1	, Found open	port nº23
[14:58:12]	IP:127.0.0.1	, Found open	port nº443
[14:58:12]	IP:127.0.0.1	, Found open	port nº21
[14:58:12]	IP:127.0.0.1	, Found open	port nº22
[14:58:13]	IP:127.0.0.1	, Found open	port nº25
[14:58:13]	IP:127.0.0.1	, Found open	port nº3389
[14:58:13]	IP:127.0.0.1	, Found open	port nº110
[15:06:02]	Checking port nº5033		

Retorno da interface durante a execução de varredura.

Capacidades Táticas: Módulos de Exploração



Reconhecimento Interno

portscan : Varredura das 8000 portas principais via SSRF.

networkscan : Ping sweep HTTP para mapear a rede interna.



Extração de Dados

readfiles : Leitura de arquivos locais como /etc/passwd.

aws / gce / alibaba : Captura de credenciais via APIs de metadados Cloud (169.254.169.254).



Execução de Código (RCE)

redis : Exploração de instâncias Redis vulneráveis via protocolo Gopher para shell reverso.

fastcgi : Execução de comandos remotos em instâncias FastCGI expostas internamente.

Arquitetura de Demonstração Local



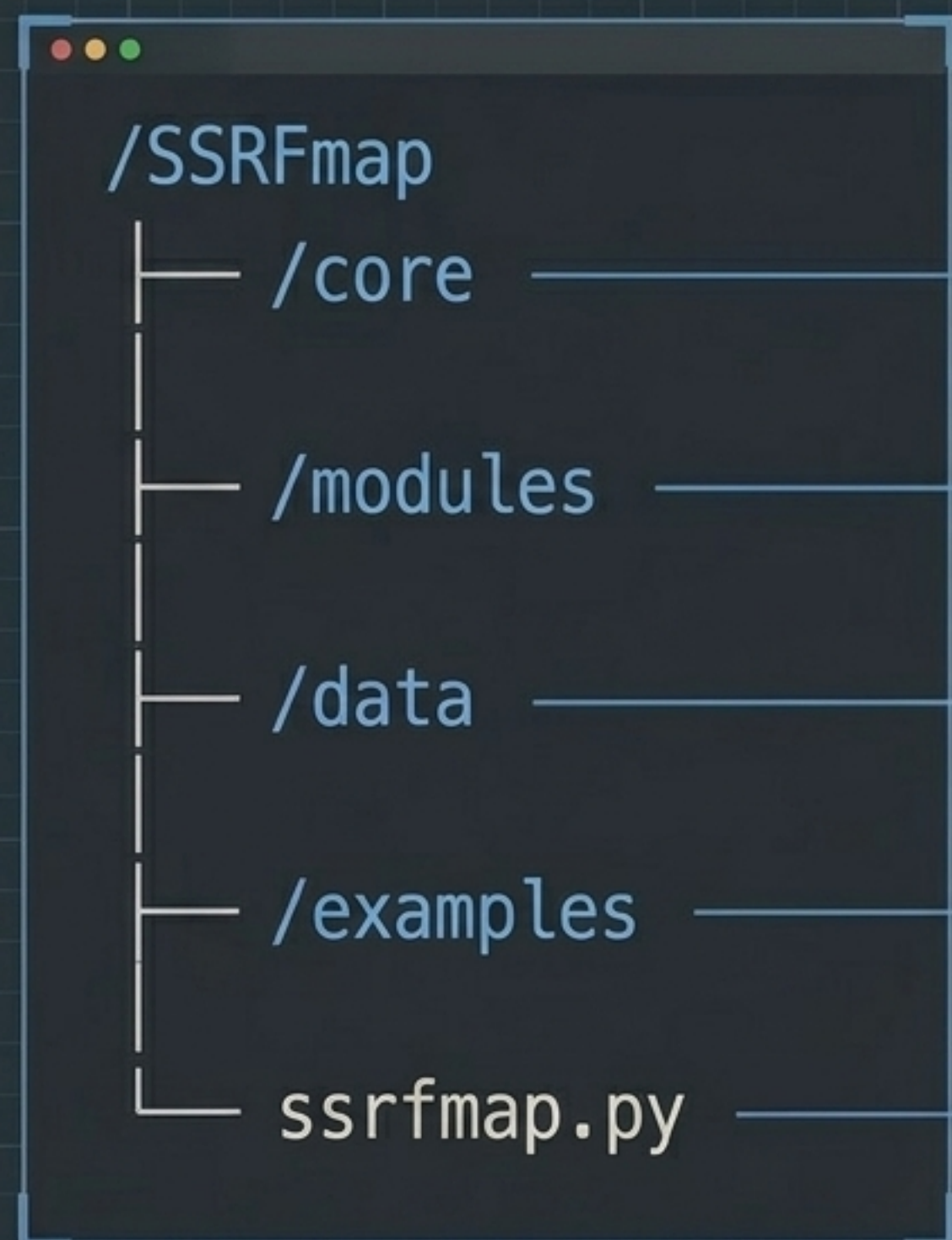
```
python3 data/example.py
```

```
python3 ssrfmap.py -r data/request2.txt  
-p url -m portscan
```

Aviso Ético: O servidor embutido é intencionalmente vulnerável. Nunca execute estes testes em ambientes públicos, de produção ou de terceiros sem autorização formal. Utilize exclusivamente em **localhost** para estudos acadêmicos e validação.

Laboratórios de Estudo Adicionais:
PortSwigger Web Security Academy | OWASP Mutillidae II

Estrutura do Projeto e Extensibilidade



Motor principal: Gerencia a manipulação de requisições HTTP, o sistema de logging e o motor base do fuzzer.

Arquitetura modular: Scripts Python isolados para cada ataque (redis, aws, etc). Desenvolvido para facilitar a contribuição acadêmica e adição de novos exploits.

Ambiente controlado: Contém o laboratório vulnerável em Flask, scripts de teste e o Dockerfile.

Templates de estudo: Arquivos de requisição base do Burp Suite devidamente formatados.

Entry-point: Ponto de entrada da aplicação na interface de linha de comando.

Análise Crítica: Comparativo de Ferramentas

Característica	SSRFmap	Burp Suite (Pro)	Gopherus	Nuclei
Foco Principal	Exploração SSRF profunda	Scanner web geral (OAST)	Geração de links Gopher	Fuzzing via Templates YAML
Geração de Payloads	Automática (Baseada no serviço alvo)	Listas estáticas e Extensões	Manual (Apenas cria o payload)	Baseada em wordlists
Requer Arquivo HTTP interceptado?	Sim (via .txt)	Não (Scanner nativo)	Não aplicável	Não (URL direta ou YAML)
Suporte OAST (Blind SSRF)	Básico e Customizado	Excelente (Burp Collaborator)	Não possui	Nativo (Interactsh)

O diferencial do SSRFmap reside na junção única entre o fuzzing automatizado do parâmetro e a geração de exploração (RCE) em linha de comando.

Vantagens Técnicas e Produtividade

Automação Acelerada



Reduz drasticamente o tempo gasto na formatação manual de payloads complexos para o protocolo Gopher e lida perfeitamente com encodings e caracteres especiais.

Evasão Integrada (Bypass)



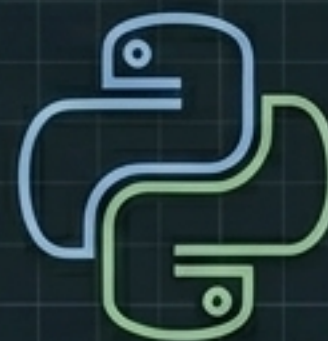
A flag `--level` itera silenciosamente sobre representações alternativas de IP (como IPv6, Hexadecimal e variações locais) para contornar Firewalls (WAFs) e filtros fracos.

Curva de Aprendizado



Adota uma sintaxe familiar e intuitiva, inspirada em padrões já estabelecidos e consolidados pela indústria de segurança ofensiva, como o SQLmap.

Código Aberto (Python 3)



Transparência total que permite auditoria acadêmica rigorosa, modificações customizadas e a fácil criação de novos módulos no diretório ``/modules``.

Limitações e Desafios Técnicos

Ausência de Crawler Nativo

Ao contrário de ferramentas abordadas em estudos acadêmicos recentes, o SSRFmap não navega autonomamente pela aplicação para descobrir a falha. A identificação prévia do parâmetro vulnerável deve ser feita manualmente pelo pesquisador.

Limitações em Blind SSRF

Em cenários de SSRF Cego (onde o servidor não retorna a resposta na tela), a ferramenta carece de integrações nativas profundas com servidores OAST robustos, tornando-se menos eficiente do que soluções comerciais nesse quesito específico.

Falsos Positivos via Latência

Como alguns módulos de varredura de rede interna dependem puramente da medição do tempo de resposta (latência), oscilações ou instabilidades de conexão na infraestrutura podem gerar falsos positivos ou resultados imprecisos durante o Portscan.

Cenários de Aplicação Prática



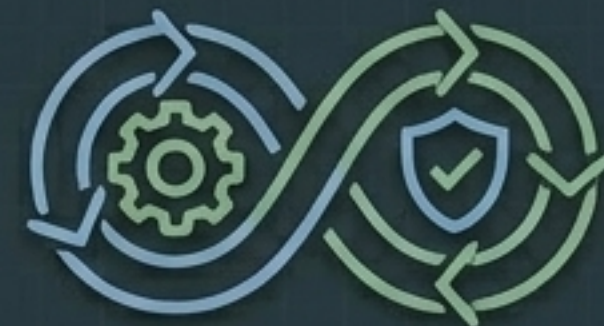
Metadados em Nuvem

Extração crítica de tokens de acesso e chaves de API temporárias por meio da exploração agressiva de APIs de metadados em ambientes AWS, GCP e Azure, resultando no comprometimento total da infraestrutura Cloud.



Pivoteamento de Rede

A aplicação web vulnerável é cooptada e transformada em uma ponte para realizar movimentação lateral, escaneando e atacando serviços administrativos internos isolados por firewalls de perímetro.



Validação em CI/CD

Uso defensivo por equipes de Blue Team e DevSecOps, automatizando testes localmente para atestar de forma empírica se novos filtros e AllowLists implementados estão bloqueando tentativas de bypass adequadamente.

Responsabilidade e Defesa Arquitetural

Rigor Ético no Pentest



- Execução estritamente restrita a escopos com autorização explícita e documentada (Contratos formais ou Bug Bounty).
- **Atenção redobrada** na utilização de módulos RCE e gravação, que podem corromper dados ou desestabilizar sistemas em produção.

Estratégias de Mitigação (Blue Team)



- **Validação Restrita (AllowLists)**: Nunca confiar no input. Implementar validação apenas contra uma lista branca de domínios pré-aprovados.
- **Bloqueio de Schemas**: Desabilitar ativamente o suporte a protocolos perigosos como file://, gopher:// e dict:// no cliente HTTP.
- **Isolamento de Rede**: Garantir que a DMZ e o servidor web não possuam rotas cegas e desnecessárias para a rede interna crítica.

Síntese e Valor Estratégico

Ascensão do Risco: O SSRF ascendeu ao OWASP Top 10 impulsionado pela proliferação maciça de arquiteturas Cloud e integrações baseadas em API.

O Catalisador da Exploração: O SSRFmap preenche ativamente a lacuna metodológica entre a simples descoberta de uma vulnerabilidade e sua exploração letal comprovada.

Encadeamento de Protocolos: A capacidade única de traduzir payloads complexos Gopher em Exploits funcionais torna a ferramenta indispensável no arsenal do Red Team.

Ferramenta Didática: Para o Blue Team, expõe de forma inegável a anatomia exata de requisições maliciosas, evidenciando a falácia de filtros de URL baseados em negação.

Referências Bibliográficas e Base de Estudo

Código e Ferramenta Original:

- SWISSKY. SSRFmap: Automatic SSRF fuzzer and exploitation tool. GitHub Repository, 2018-2024.

Literatura Científica e Artigos:

- CHEN, Jianjun et al. Where URLs Become Weapons: Automated Discovery of SSRF Vulnerabilities in Web Applications. IEEE Symposium on Security and Privacy, 2024.

Metodologia e Arquitetura de Testes:

- ALMEIDA JR., José Augusto de. Pentest Em Aplicações Web. Editora Casa do Código.

Padrões Globais de Segurança e Defesa:

- OWASP. Server-Side Request Forgery Prevention Cheat Sheet. The Open Web Application Security Project.

Casos de Estudo e Mídia:

- SOLYD OFFENSIVE SECURITY. Como explorar SSRF – Pentest / Bug Bounty.
- VAADATA. Exploiting SSRF vulnerability: Server-Side Request Forgery.